

Lab 3: Monitoring, Logging & DevSecOps

Session 1 - Question 1: Set up monitoring using Prometheus and Grafana (5 marks)

Name	
Roll No. / Reg. No.	
Course	DevOps
Date of Experiment	06-10-2026
Tools Used	Docker Compose, Prometheus v2.55.1, Grafana 11.3.0, Node Exporter v1.8.2, Python 3.12 (prometheus_client), Windows 11

Problem Statement

Set up monitoring using Prometheus and visualize metrics using Grafana dashboards for a target application or system.

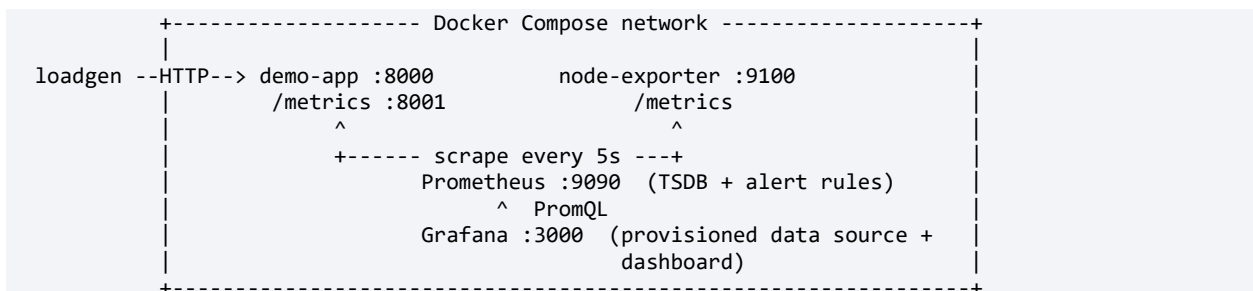
Aim

To set up Prometheus to collect metrics from a target application and from the host system, and to visualize those metrics on Grafana dashboards.

Theory

- **Prometheus:** an open-source monitoring system that pulls (scrapes) metrics over HTTP from targets at a fixed interval, stores them as time series and lets you query them with PromQL.
- **Exporters / instrumentation:** targets expose metrics on a /metrics endpoint. Node Exporter exposes host CPU, memory, disk and network metrics; the demo app exposes its own metrics using the prometheus_client library.
- **Metric types:** Counter (only increases, e.g. requests), Gauge (goes up and down, e.g. requests in progress), Histogram (distribution, e.g. latency buckets).
- **Grafana:** a visualization tool that connects to Prometheus as a data source and shows PromQL queries as graphs, stats and dashboards.
- **Alert rules:** PromQL conditions evaluated by Prometheus, e.g. error rate > 5% or a target down.

Architecture



Component	Image	Port	Purpose
demo-app	python:3.12-alpine + prometheus_client	8000 / 8001	Target application; exposes app_requests_total,

			app_request_duration_seconds, app_requests_in_progress, app_orders_total
loadgen	busybox:1.36	-	Sends continuous traffic (/ , /order, /missing) so metrics change
node-exporter	prom/node-exporter:v1.8.2	9100	System metrics (CPU, memory, network)
prometheus	prom/prometheus:v2.55.1	9090	Scrapes all targets every 5 s, evaluates alert rules
grafana	grafana/grafana:11.3.0	3000	Dashboards; data source and dashboard provisioned from files

Configuration Files

docker-compose.yml

```

services:
  demo-app:
    build: ./app
    container_name: demo-app
    ports:
      - "8000:8000"
      - "8001:8001"

  loadgen:
    image: busybox:1.36
    container_name: loadgen
    command: ["sh", "/loadgen.sh"]
    volumes:
      - ./app/loadgen.sh:/loadgen.sh:ro
    depends_on: [demo-app]

  node-exporter:
    image: prom/node-exporter:v1.8.2
    container_name: node-exporter
    ports:
      - "9100:9100"

  prometheus:
    image: prom/prometheus:v2.55.1
    container_name: prometheus
    volumes:
      - ./prometheus:/etc/prometheus:ro
      - prometheus-data:/prometheus
    ports:
      - "9090:9090"
    depends_on: [demo-app, node-exporter]

  grafana:
    image: grafana/grafana:11.3.0
    container_name: grafana
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin123
      - GF_AUTH_ANONYMOUS_ENABLED=true
      - GF_AUTH_ANONYMOUS_ORG_ROLE=Viewer
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
      - grafana-data:/var/lib/grafana
    ports:
      - "3000:3000"
    depends_on: [prometheus]

volumes:
  prometheus-data:

```

```
grafana-data:
```

prometheus/prometheus.yml

```
global:
  scrape_interval: 5s
  evaluation_interval: 5s

rule_files:
  - /etc/prometheus/alert-rules.yml

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: node-exporter          # host / system metrics
    static_configs:
      - targets: ["node-exporter:9100"]

  - job_name: demo-app              # application metrics
    static_configs:
      - targets: ["demo-app:8001"]
```

prometheus/alert-rules.yml

```
groups:
  - name: demo-app-alerts
    rules:
      - alert: HighErrorRate
        expr: sum(rate(app_requests_total{status=~"5.."}[1m])) / sum(rate(app_requests_total[1m]))
        > 0.05
        for: 30s
        labels:
          severity: warning
        annotations:
          summary: "More than 5% of requests are failing"
      - alert: TargetDown
        expr: up == 0
        for: 30s
        labels:
          severity: critical
        annotations:
          summary: "Target {{ $labels.job }} is down"
```

grafana/provisioning/datasources/prometheus.yml

```
apiVersion: 1
datasources:
  - name: Prometheus
    uid: prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
```

Instrumented application (app/app.py)

```
"""Demo web app instrumented with Prometheus metrics (prometheus_client)."""
import random
import time
from http.server import BaseHTTPRequestHandler, HTTPServer

from prometheus_client import Counter, Gauge, Histogram, start_http_server
```

```

REQUESTS = Counter("app_requests_total", "Total HTTP requests", ["endpoint", "status"])
LATENCY = Histogram("app_request_duration_seconds", "Request latency in seconds", ["endpoint"],
                    buckets=(0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0))
IN_PROGRESS = Gauge("app_requests_in_progress", "Requests currently being processed")
ORDERS = Counter("app_orders_total", "Business metric: orders placed")

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        endpoint = self.path.split("?")[0]
        IN_PROGRESS.inc()
        start = time.time()
        if endpoint == "/order":
            time.sleep(random.uniform(0.05, 0.3))
            status = 500 if random.random() < 0.1 else 200
            if status == 200:
                ORDERS.inc()
        elif endpoint == "/":
            time.sleep(random.uniform(0.005, 0.05))
            status = 200
        else:
            status = 404
        LATENCY.labels(endpoint).observe(time.time() - start)
        REQUESTS.labels(endpoint, str(status)).inc()
        IN_PROGRESS.dec()
        body = f"status={status}\n".encode()
        self.send_response(status)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def log_message(self, *args):
        pass

if __name__ == "__main__":
    start_http_server(8001)          # /metrics endpoint for Prometheus
    print("App on :8000, metrics on :8001/metrics", flush=True)
    HTTPServer(("0.0.0.0", 8000), Handler).serve_forever()

```

Procedure / Commands

1. Wrote the demo application with Prometheus instrumentation and its Dockerfile.
2. Wrote prometheus.yml with three scrape jobs (prometheus, node-exporter, demo-app) and alert rules.
3. Wrote Grafana provisioning files so the Prometheus data source and the dashboard are created automatically.
4. Started the stack: docker compose up -d --build
5. Verified the targets in Prometheus (Status > Targets) and ran PromQL queries.
6. Opened Grafana (<http://localhost:3000>, admin / admin123) and viewed the 'Lab 3 - Application & System Monitoring' dashboard.

```

docker compose up -d --build
docker compose ps
curl http://localhost:8001/metrics
curl http://localhost:9090/api/v1/targets
# Prometheus UI : http://localhost:9090
# Grafana UI    : http://localhost:3000

```

Output / Screenshots

```
MINGW64 - Git Bash

pav@laptop MINGW64 /c/Users/pav/mon-lab
$ docker --version
Docker version 29.3.1, build c2be9cc

pav@laptop MINGW64 /c/Users/pav/mon-lab
$ docker compose version
Docker Compose version v5.1.1

pav@laptop MINGW64 /c/Users/pav/mon-lab
$ docker compose images
```

CONTAINER	REPOSITORY	TAG	PLATFORM	IMAGE ID	SIZE	CREATED
demo-app	mon-lab-demo-app	latest	linux/amd64	484c92b492e9	24.2MB	2 minutes ago
grafana	grafana/grafana	11.3.0	linux/amd64	a0f881232a6f	133MB	23 months ago
loadgen	busybox	1.36	linux/amd64	73aaf090f3d8	2.22MB	3 years ago
node-exporter	prom/node-exporter	v1.8.2	linux/amd64	4032c6d5bfd7	12MB	2 years ago
prometheus	prom/prometheus	v2.55.1	linux/amd64	2659f4c2ebb7	114MB	23 months ago

Figure 1.1: Docker / Compose versions and images used

```
MINGW64 - Git Bash

pav@laptop MINGW64 /c/Users/pav/mon-lab
$ docker compose ps --format 'table {{.Name}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
```

NAME	IMAGE	STATUS	PORTS
demo-app	mon-lab-demo-app	Up 2 minutes	0.0.0.0:8000-8001->8000-8001/tcp, [::]:8000-8001->8000-8001/tcp
grafana	grafana/grafana:11.3.0	Up 2 minutes	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
loadgen	busybox:1.36	Up 2 minutes	
node-exporter	prom/node-exporter:v1.8.2	Up 2 minutes	0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp
prometheus	prom/prometheus:v2.55.1	Up 2 minutes	0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp

Figure 1.2: All five containers running

```
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 250.0
python_gc_objects_collected_total{generation="1"} 12.0
python_gc_objects_collected_total{generation="2"} 0.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 33.0
python_gc_collections_total{generation="1"} 3.0
python_gc_collections_total{generation="2"} 0.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="12",patchlevel="15",version="3.12.15"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 3.18464e+07
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 2.302712e+07
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.79127559452e+09
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 12.34
# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 9.0
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1.048576e+06
# HELP app_requests_total Total HTTP requests
# TYPE app_requests_total counter
app_requests_total{endpoint="/",status="200"} 297.0
app_requests_total{endpoint="/order",status="200"} 264.0
app_requests_total{endpoint="/missing",status="404"} 53.0
app_requests_total{endpoint="/order",status="500"} 33.0
# HELP app_requests_created Total HTTP requests
# TYPE app_requests_created gauge
app_requests_created{endpoint="/",status="200"} 1.791275603955858e+09
app_requests_created{endpoint="/order",status="200"} 1.7912756042171788e+09
app_requests_created{endpoint="/missing",status="404"} 1.7912756043472848e+09
app_requests_created{endpoint="/order",status="500"} 1.791275605102749e+09
# HELP app_request_duration_seconds Request latency in seconds
# TYPE app_request_duration_seconds histogram
app_request_duration_seconds_bucket{endpoint="/",le="0.01"} 30.0
app_request_duration_seconds_bucket{endpoint="/",le="0.025"} 124.0
app_request_duration_seconds_bucket{endpoint="/",le="0.05"} 283.0
app_request_duration_seconds_bucket{endpoint="/",le="0.1"} 294.0
app_request_duration_seconds_bucket{endpoint="/",le="0.25"} 297.0
app_request_duration_seconds_bucket{endpoint="/",le="0.5"} 297.0
app_request_duration_seconds_bucket{endpoint="/",le="1.0"} 297.0
app_request_duration_seconds_bucket{endpoint="/",le="Inf"} 297.0
app_request_duration_seconds_count{endpoint="/"} 297.0
```

Figure 1.3: Raw metrics exposed by the demo app at /metrics

```
MINGW64 - Git Bash

pav@laptop MINGW64 /c/Users/pav/mon-lab
$ curl -s localhost:9090/api/v1/targets | python -c "import json,sys;[print(t['labels']['job'].ljust(15), t['scrapeUrl'].ljust(35), t['health']) for t in json.load(sys.stdin)['data']['activeTargets']]"
demo-app      http://demo-app:8001/metrics      up
node-exporter  http://node-exporter:9100/metrics  up
prometheus     http://localhost:9090/metrics      up

pav@laptop MINGW64 /c/Users/pav/mon-lab
$ curl -s localhost:8001/metrics | grep -E '^app_' | head -14
app_requests_total{endpoint="/",status="200"} 226.0
app_requests_total{endpoint="/order",status="200"} 199.0
app_requests_total{endpoint="/missing",status="404"} 42.0
app_requests_total{endpoint="/order",status="500"} 26.0
app_requests_created{endpoint="/",status="200"} 1.7912756039558558e+09
app_requests_created{endpoint="/order",status="200"} 1.7912756042171788e+09
app_requests_created{endpoint="/missing",status="404"} 1.7912756043472848e+09
app_requests_created{endpoint="/order",status="500"} 1.791275605102749e+09
app_request_duration_seconds_bucket{endpoint="/",le="0.01"} 23.0
app_request_duration_seconds_bucket{endpoint="/",le="0.025"} 96.0
app_request_duration_seconds_bucket{endpoint="/",le="0.05"} 218.0
app_request_duration_seconds_bucket{endpoint="/",le="0.1"} 225.0
app_request_duration_seconds_bucket{endpoint="/",le="0.25"} 226.0
app_request_duration_seconds_bucket{endpoint="/",le="0.5"} 226.0
```

Figure 1.4: Targets and sample application metrics from the command line

 Prometheus Alerts Graph Status Help

Targets

All scrape pools

All Unhealthy Collapse All

Filter by endpoint or labels

Unknown Unhealthy Healthy

demo-app (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://demo-app:8001/metrics	UP	instance="demo-app:8001" job="demo-app"	921.000ms ago	64.005ms	

node-exporter (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://node-exporter:9100/metrics	UP	instance="node-exporter:9100" job="node-exporter"	1.227s ago	107.279ms	

prometheus (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	5.42s ago	314.268ms	

Figure 1.5: Prometheus Targets page - demo-app, node-exporter and prometheus are all UP

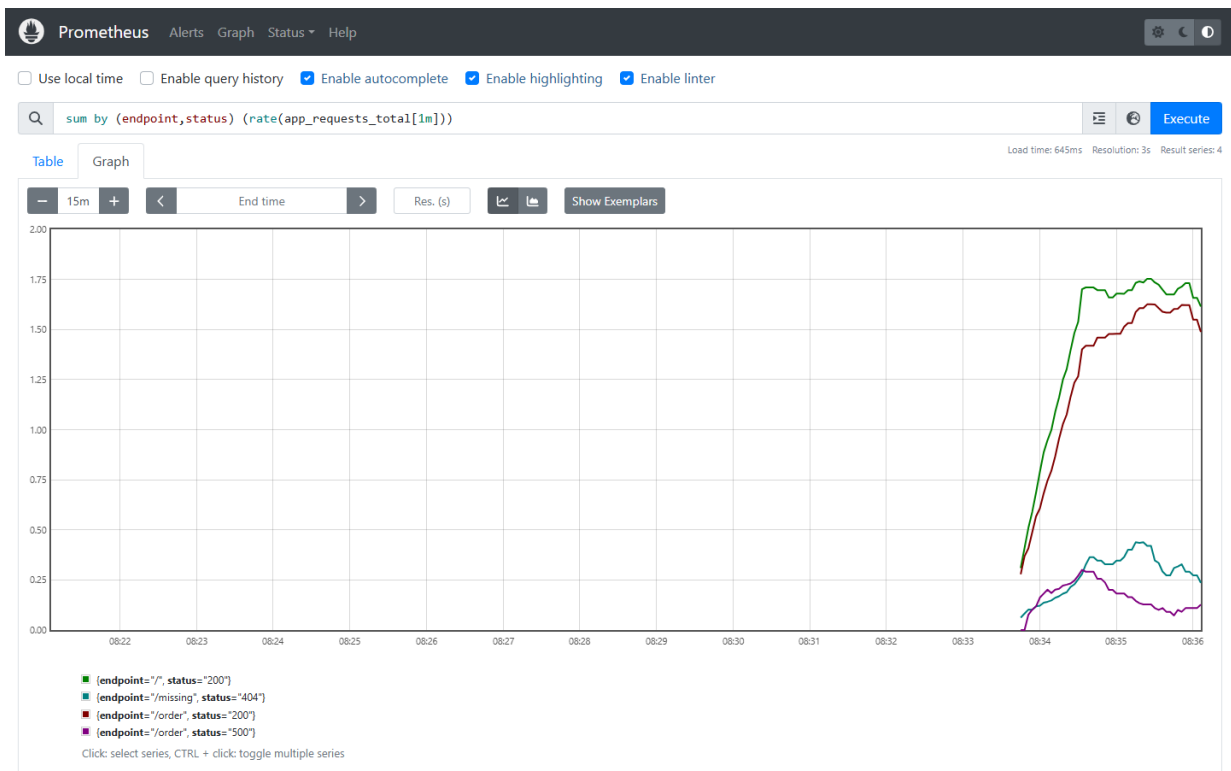


Figure 1.6: PromQL: request rate by endpoint and status

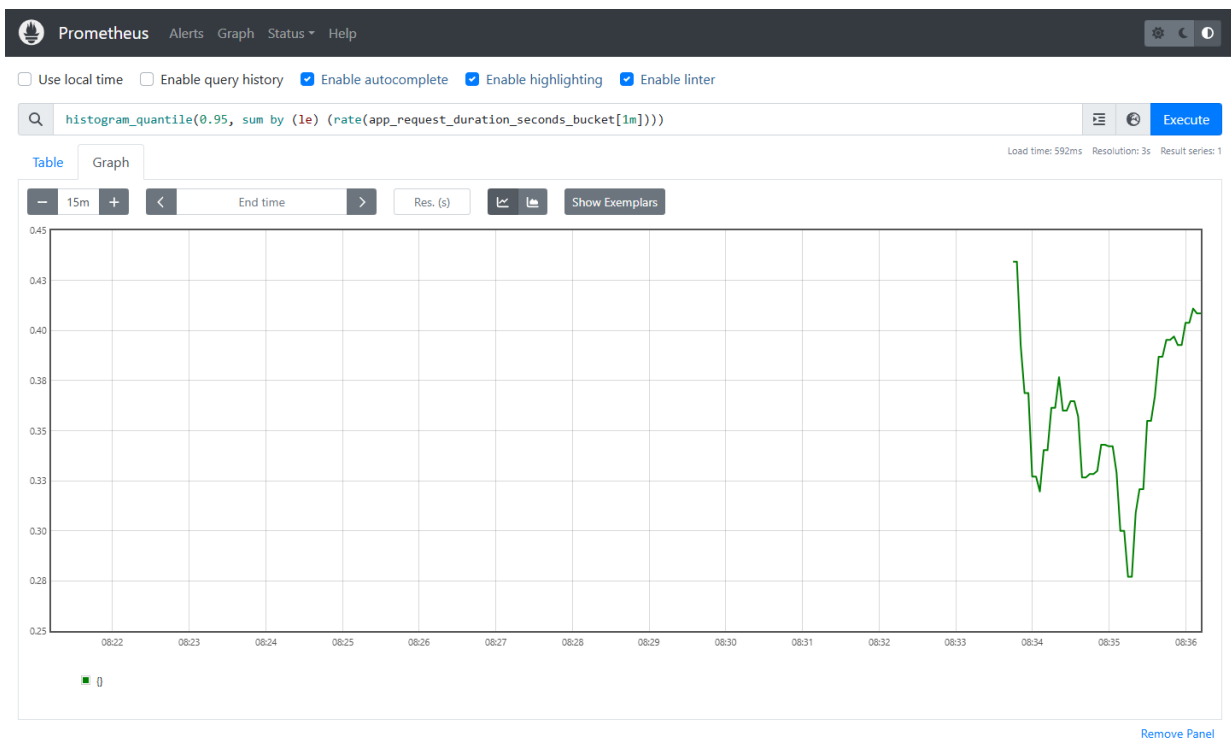


Figure 1.7: PromQL: 95th percentile latency from the histogram

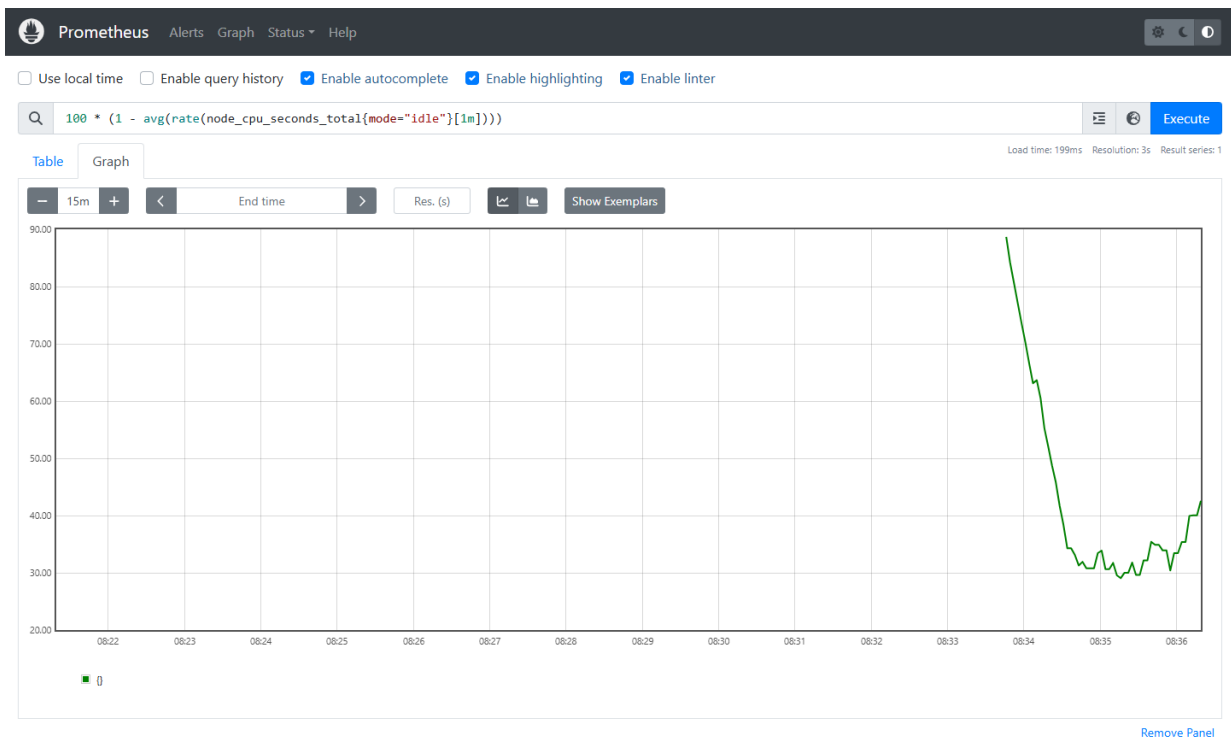


Figure 1.8: PromQL: host CPU usage from node-exporter

```

MINGW64 - Git Bash
pav@laptop MINGW64 /c/Users/pav/mon-lab
$ curl -s 'localhost:9090/api/v1/query' --data-urlencode 'query=sum by (endpoint,status) (rate(app_requests_total[1m]))' | python -m json.tool | grep -E 'endpoint|status'|"[0-9.]+"' | head -24
  "status": "success",
    "endpoint": "/",
    "status": "200"
    "1.6901502727668263"
    "endpoint": "/order",
    "status": "200"
    "1.6064794671843101"
    "endpoint": "/missing",
    "status": "404"
    "0.2677465778640517"
    "endpoint": "/order",
    "status": "500"
    "0.10040496669901938"

pav@laptop MINGW64 /c/Users/pav/mon-lab
$ curl -s 'localhost:9090/api/v1/query' --data-urlencode 'query=100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"})[1m]))' | python -c "import json,sys;print('Host CPU %:', json.load(sys.stdin)['data']['result'][0]['value'][1])"
Host CPU %: 35.41126410752755

```

Figure 1.9: PromQL queries through the HTTP API

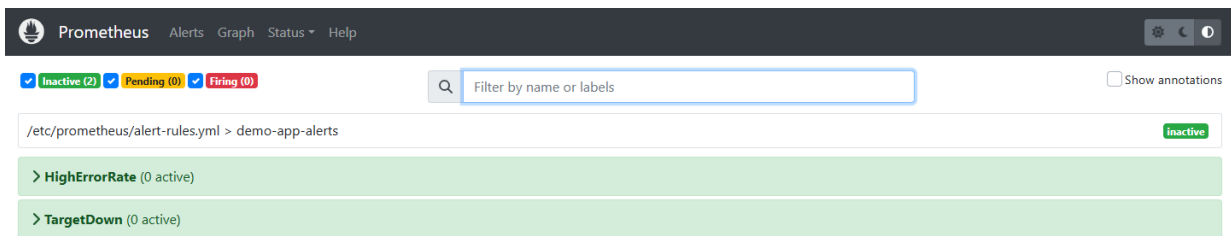


Figure 1.10: Alert rules loaded in Prometheus (HighErrorRate, TargetDown)

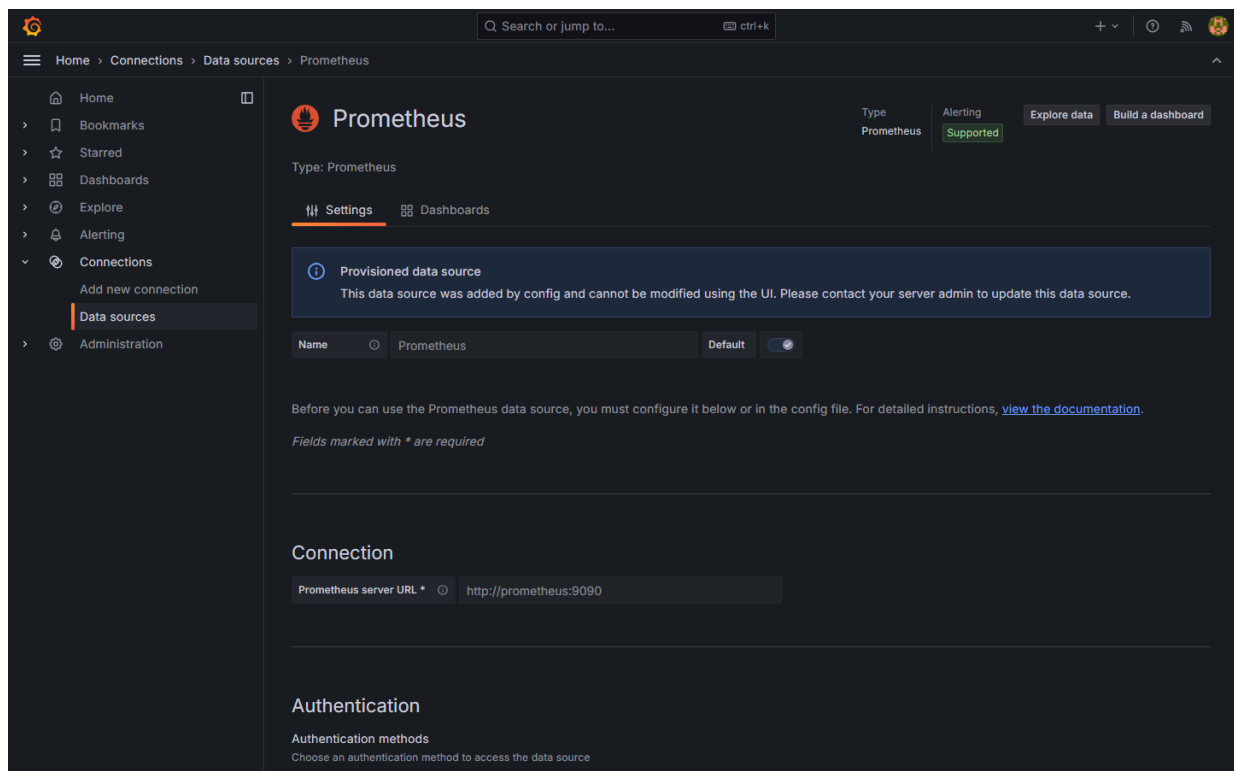


Figure 1.11: Grafana - Prometheus data source (provisioned)

```
MINGW64 - Git Bash

pav@laptop MINGW64 /c/Users/pav/mon-lab
$ curl -s localhost:3000/api/health
{
  "database": "ok",
  "version": "11.3.0",
  "commit": "d9455ff7db73b694db7d412e49a68bec767f2b5a"
}

pav@laptop MINGW64 /c/Users/pav/mon-lab
$ curl -s -u admin:admin123 localhost:3000/api/datasources | python -m json.tool | grep -E 'name|type'|url'
  "name": "Prometheus",
  "type": "prometheus",
  "url": "http://prometheus:9090",

pav@laptop MINGW64 /c/Users/pav/mon-lab
$ curl -s -u admin:admin123 'localhost:3000/api/search?query=Lab' | python -m json.tool | grep -E 'title|url'
  "title": "Lab3",
  "url": "/dashboards/f/bg0e7090frqwwa/lab3",
  "title": "Lab 3 - Application & System Monitoring",
  "url": "/d/lab3-demo/lab-3-application-and-system-monitoring",
```

Figure 1.12: Grafana health, data source and dashboard via the API

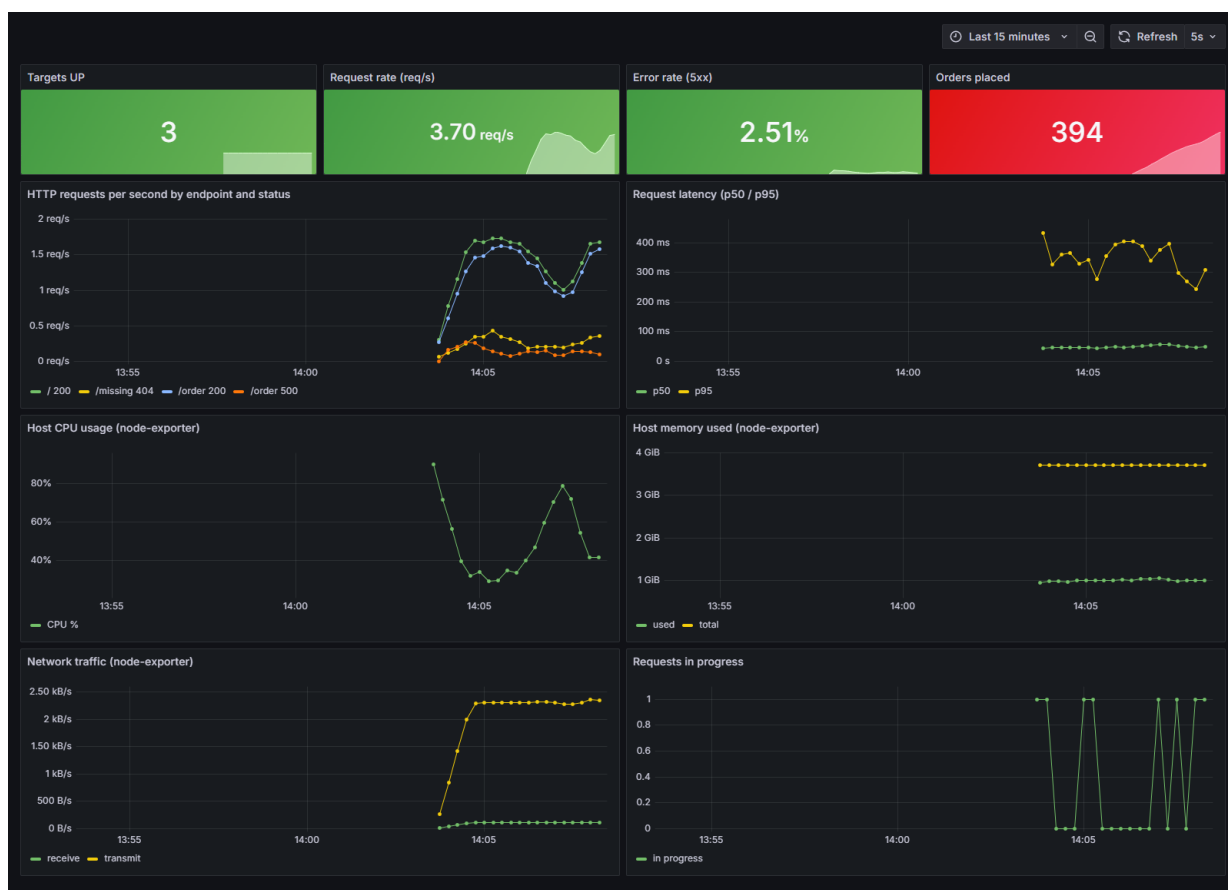


Figure 1.13: Grafana dashboard - application and system metrics (live)

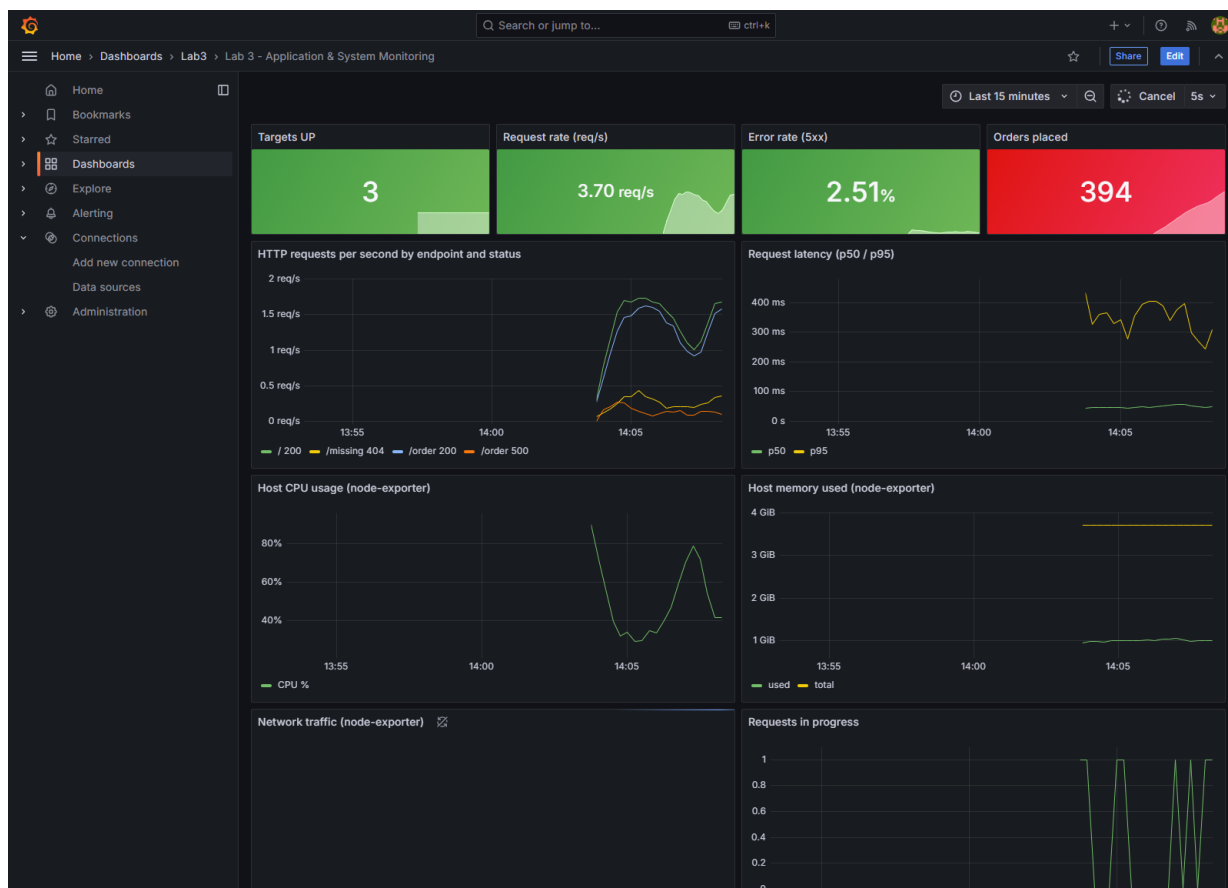


Figure 1.14: Dashboard inside the Grafana UI

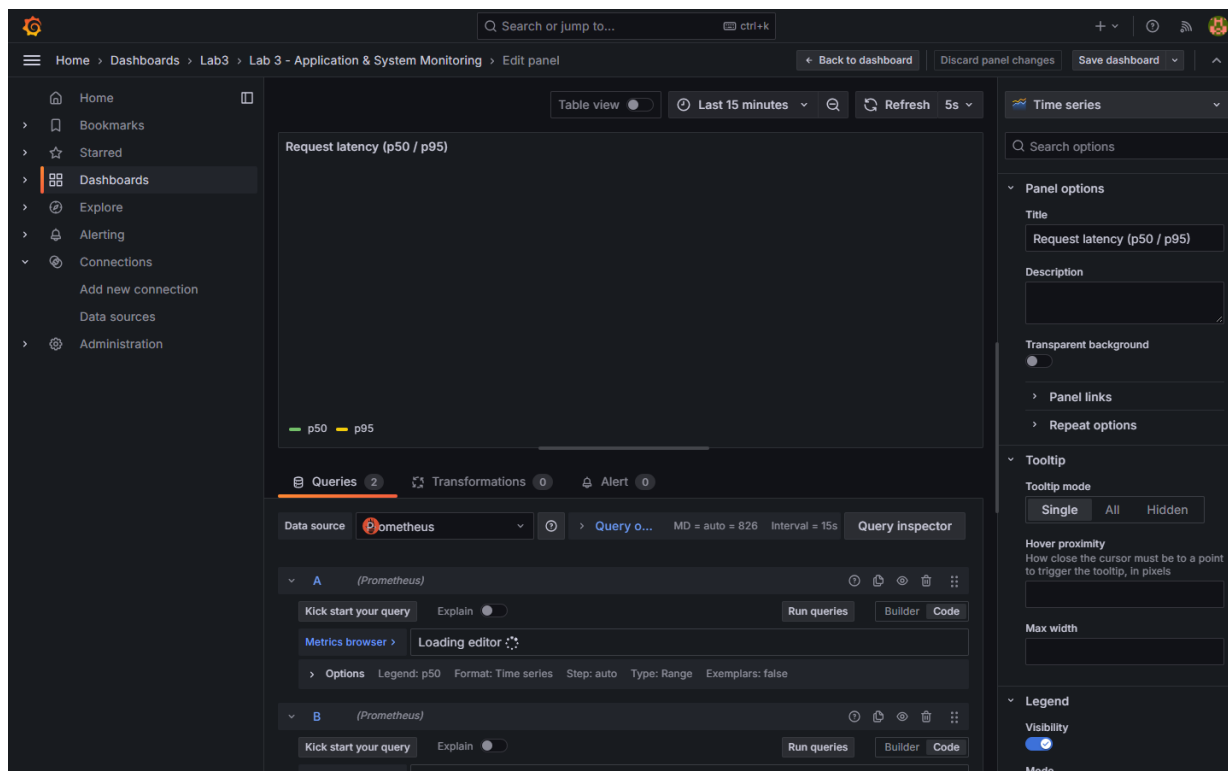


Figure 1.15: Editing a panel - the PromQL query behind the latency graph

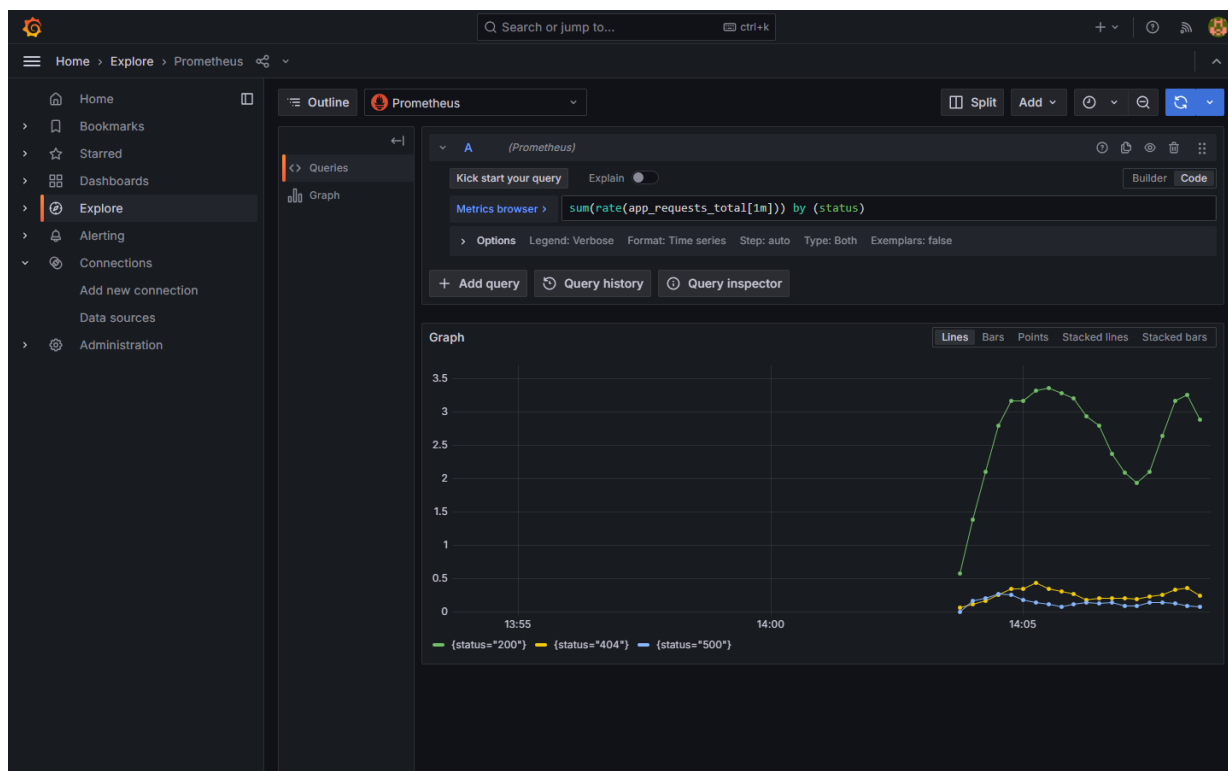


Figure 1.16: Grafana Explore - ad-hoc PromQL query

Dashboard Panels

Panel	PromQL
Targets UP	sum(up)
Request rate	sum(rate(app_requests_total[1m]))
Error rate (5xx)	sum(rate(app_requests_total{status=~"5.."}[1m])) / sum(rate(app_requests_total[1m]))
Orders placed	sum(app_orders_total)
Requests by endpoint/status	sum by (endpoint,status) (rate(app_requests_total[1m]))
Latency p50 / p95	histogram_quantile(0.95, sum by (le) (rate(app_request_duration_seconds_bucket[1m])))
Host CPU %	100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[1m])))
Host memory used	node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes
Network traffic	sum(rate(node_network_receive_bytes_total[1m])), ..._transmit_...
Requests in progress	app_requests_in_progress

Results

- All 3 scrape targets were UP; Prometheus scraped them every 5 seconds.
- The dashboard showed about 3.7 requests/s, an error rate of about 2.5% (the /order endpoint returns HTTP 500 for ~10% of calls), p95 latency of about 300-400 ms on /order, and 394 orders placed at the time of the screenshot.
- Host metrics (CPU, memory around 1 GiB used of about 3.7 GiB, network traffic) were visible from node-exporter.

Observations

- Prometheus uses a pull model: adding a new target only needs a `scrape_config` entry.
- Counters must be viewed with `rate()` to get per-second values; histograms with `histogram_quantile()` to get percentiles.
- Provisioning the data source and dashboard as files makes the monitoring setup reproducible (monitoring as code).
- The p95 latency is much higher than p50, which an average alone would hide - percentiles are important for user experience.
- Error-rate and target-down alert rules turn the metrics into actionable alerts.

Conclusion

Prometheus was set up to monitor a containerized application and the host system, and Grafana dashboards were built to visualize request rate, errors, latency and system resources in real time.